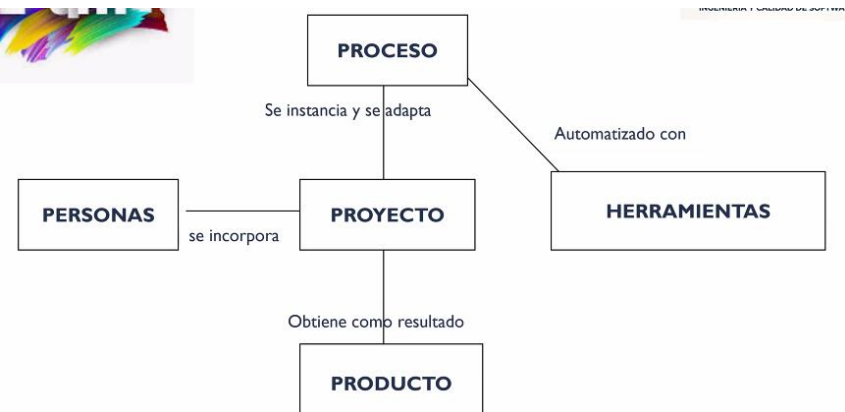


COMPONENTES DE PROYECTO DE DESARROLLO DE SOFTWARE

El proceso de software: conjunto de actividades para desarrollar un sistema de software. Van dependiendo de la organización y el tipo de sistema que debe desarrollarse. Debe ser explícitamente modelado su va a ser administrado.



El Proceso (que define el "cómo") se instancia y se adapta a las necesidades del Proyecto en particular.

Al Proyecto se le incorporan las Personas con sus respectivas habilidades y motivación.

El equipo de personas ejecuta las tareas del proyecto utilizando y automatizando los pasos del proceso mediante Herramientas.

Como resultado final de esta articulación, se obtiene el Producto de software que aporta valor al cliente.

El proyecto es una unidad de gestión del trabajo y es el medio por el que obtenemos un producto. La ingeniería de software da saberes para todos estos aspectos. Las personas que se suman a los proyectos son un elemento fundamental para que sea exitoso.

Proceso de software:

El desarrollo de software no puede abordarse de la misma manera que la fabricación de un producto tangible. A diferencia de objetos como un control remoto, una pantalla o cualquier producto elaborado mediante una línea de producción, el software no se produce a través de un proceso completamente repetitivo y masivo. Esta característica constituye la base para comprender los distintos enfoques utilizados en la gestión y el desarrollo de proyectos de software.

Cada enfoque adopta una perspectiva diferente respecto de cómo realizar estimaciones, qué métricas considerar importantes, qué aspectos planificar, cómo y cuándo planificarlos, cómo gestionar los cambios en los requerimientos y de qué manera utilizar la experiencia obtenida durante los proyectos. A partir de estas diferencias pueden distinguirse principalmente los procesos definidos y los procesos empíricos.

Procesos definidos

Los procesos definidos parten de la idea de que es posible establecer previamente, y con cierto nivel de detalle, las actividades necesarias para desarrollar software. En ellos pueden determinarse las entradas y salidas de las actividades, las métricas utilizadas, los roles responsables y las tareas que deberían realizarse durante el proyecto.

Sin embargo, un proceso definido representa una descripción teórica de aquello que debería realizarse. Por este motivo, cuando se aplica a un proyecto concreto debe atravesar un proceso de instanciación y adaptación. Esto significa analizar el proceso general y determinar qué partes resultan necesarias según las características particulares del producto, el equipo de trabajo, los clientes y el contexto del proyecto.

Esta adaptación no implica que las actividades establecidas por la organización puedan eliminarse arbitrariamente. Por ejemplo, si el proceso organizacional establece que debe documentarse la arquitectura del sistema, la decisión de no hacerlo debe estar correctamente fundamentada de acuerdo con las características específicas del proyecto.

Uno de los principales objetivos de los procesos definidos es alcanzar cierto grado de repetibilidad y previsibilidad. Se considera que la experiencia obtenida en proyectos anteriores puede utilizarse como referencia para proyectos futuros. Esto permite aprovechar información histórica para realizar estimaciones relacionadas con costos, tiempos, recursos y actividades.

Dentro de los enfoques tradicionales existen técnicas de estimación que sistematizan la información obtenida de proyectos anteriores y la convierten en datos históricos. Estos datos permiten tomar como referencia la experiencia previa al momento de estimar un nuevo proyecto.

Sin embargo, utilizar experiencias anteriores no garantiza que los mismos resultados vuelvan a producirse. Lo que funcionó para un equipo o proyecto puede no funcionar de la misma manera en otro contexto. Por lo tanto, la experiencia previa funciona como una referencia y no como una certeza.

Además, la repetibilidad de un proceso definido no significa que frente a una misma entrada se obtendrá siempre exactamente la misma salida. En el desarrollo de software intervienen factores humanos, técnicos y contextuales que dificultan la obtención de resultados idénticos. La finalidad es aumentar la previsibilidad y establecer una forma organizada de trabajo, no garantizar resultados exactamente iguales.

Procesos empíricos

Los procesos empíricos parten de una concepción diferente. Consideran que la experiencia obtenida por un equipo dentro de un proyecto no necesariamente puede trasladarse directamente a otro proyecto o equipo. Cada situación posee características propias, por lo que el conocimiento se genera principalmente a través de la experiencia directa.

En lugar de basarse fundamentalmente en datos históricos de otros proyectos, el equipo genera su propia experiencia mediante iteraciones frecuentes y entregas de software funcionando. A medida que se desarrollan estas iteraciones, se obtiene información real sobre la capacidad del equipo, sus tiempos de trabajo, sus dificultades y los resultados que puede alcanzar.

Esta información es utilizada posteriormente para realizar nuevas estimaciones, tomar decisiones y asumir compromisos. De esta manera, el equipo construye progresivamente una experiencia propia basada en lo que efectivamente ocurre durante el proyecto.

Tratamiento de los cambios: Una de las diferencias más importantes entre los procesos definidos y los procesos empíricos se encuentra en la actitud que adoptan frente a los cambios.

En los procesos empíricos se busca obtener retroalimentación lo antes posible. Para ello se intenta generar rápidamente una parte del producto que pueda mostrarse al cliente. El objetivo es recibir feedback temprano y comprobar si lo desarrollado responde realmente a sus necesidades.

Si el cliente indica que lo realizado no resulta adecuado, el equipo puede modificarlo o descartarlo antes de haber invertido una gran cantidad de tiempo y recursos. Por esta razón, los procesos empíricos trabajan mediante ciclos cortos de desarrollo, evaluación y adaptación.

Los procesos definidos, en cambio, tienden a realizar una mayor planificación previa y a utilizar procesos establecidos como guía para el desarrollo. Los cambios pueden incorporarse, pero se analizan dentro de una estructura previamente definida y considerando su impacto sobre la planificación existente.

Por lo tanto, la principal diferencia no consiste en que un enfoque permita cambios y el otro no, sino en la manera en que cada uno los enfrenta. El enfoque empírico busca detectar y responder rápidamente a los cambios mediante iteraciones y feedback frecuente, mientras que el enfoque definido intenta gestionarlos dentro de una planificación y un proceso previamente establecidos.

Diferencia respecto de la experiencia: Otra diferencia central se encuentra en la utilización de la experiencia.

Los procesos definidos consideran que la información obtenida en proyectos anteriores puede ser aprovechada como referencia para proyectos futuros. La organización puede recopilar datos históricos y utilizarlos para mejorar sus estimaciones y aumentar su capacidad de predicción.

Los procesos empíricos, en cambio, priorizan la experiencia que el propio equipo genera dentro del proyecto. El equipo desarrolla progresivamente conocimiento sobre su capacidad de trabajo a través de iteraciones frecuentes y resultados concretos.

En consecuencia, los procesos definidos buscan aprovechar la experiencia histórica para obtener mayor previsibilidad, mientras que los procesos empíricos buscan generar rápidamente experiencia propia para aprender, adaptarse y responder a las condiciones reales del proyecto.

Tipos de ciclo de vida

Existen tres tipos genéricos de ciclo de vida para el desarrollo de software:

- Secuencial.
- Iterativo o incremental.
- Recursivo.

A partir de estos tipos generales se construyen distintos ciclos de vida concretos. Es importante distinguir entre el tipo de ciclo de vida y el ciclo de vida específico.

Por ejemplo, dentro de los ciclos de vida de tipo secuencial se encuentra el modelo en cascada. En cambio, dentro de los ciclos de vida de tipo iterativo se encuentra el ciclo iterativo e incremental propuesto por el Proceso Unificado.

Por lo tanto:

- Tipo de ciclo de vida: categoría general.
- Ciclo de vida: forma concreta de organizar el desarrollo siguiendo uno de esos tipos.

Ciclo de vida secuencial

En un ciclo de vida secuencial, las actividades se realizan siguiendo un orden determinado. Generalmente se completa una actividad antes de pasar a la siguiente. El ejemplo más representativo es el ciclo de vida en cascada. En este modelo se busca completar, por ejemplo, el 100 % de los requerimientos antes de avanzar al análisis, luego completar el análisis antes de avanzar al diseño, y así sucesivamente. La estructura sería aproximadamente:

Requerimientos → Análisis → Diseño → Implementación → Pruebas.

Ciclo de vida iterativo e incremental

En un ciclo de vida iterativo no se intenta desarrollar todo el producto en una única pasada. Se selecciona una parte de la funcionalidad y se realizan sobre ella las diferentes actividades del proceso. De esta manera, se atraviesan requerimientos, análisis, diseño, implementación y pruebas sobre una cantidad limitada de funcionalidades. Como resultado se obtiene una versión parcial o incremento del producto. Luego se realiza una nueva iteración incorporando nuevas funcionalidades o modificando las existentes. Esto permite obtener rápidamente software funcionando para que pueda ser evaluado por el usuario final y generar retroalimentación. El Proceso Unificado, por ejemplo, propone utilizar un ciclo de vida iterativo e incremental.

Ciclo de vida recursivo

En un ciclo de vida recursivo las actividades o etapas pueden volver a ejecutarse reiteradamente cuando resulta necesario. Existe la posibilidad de regresar sobre actividades anteriores y volver a desarrollarlas. Este tipo de ciclo de vida también puede utilizarse dentro de procesos definidos.

Diferencia entre proceso y ciclo de vida

El proceso y el ciclo de vida son conceptos relacionados, pero no representan lo mismo. El proceso define teóricamente qué debe hacerse para desarrollar software. Especifica actividades, entradas, salidas, roles y otros elementos necesarios para realizar el trabajo. Los roles se definen de manera genérica porque posteriormente son asumidos por personas concretas dentro de cada proyecto. Una misma persona puede incluso asumir uno o varios roles dependiendo del tamaño y las características de la organización. Por lo tanto:

Proceso = define qué hay que hacer para desarrollar software.

El ciclo de vida, en cambio, determina cómo se utilizará ese proceso dentro de un proyecto específico. Establece cuándo se realizan las actividades, cuánto se realiza de cada una en cada momento y cuáles serán los puntos de control. Por lo tanto:

Ciclo de vida = determina cómo y cuándo se utilizan las actividades definidas por el proceso.

Un mismo proceso puede utilizarse con distintos ciclos de vida. Sin embargo, la elección incorrecta del ciclo de vida puede generar problemas aunque el proceso utilizado sea adecuado.

Por ejemplo, el Proceso Unificado recomienda un ciclo de vida iterativo e incremental. Si se utiliza el Proceso Unificado pero se organizan sus actividades mediante un ciclo de vida en cascada, se está utilizando el mismo proceso de una forma diferente a la recomendada.

Ejemplo de la diferencia

Puede suponerse que un proceso establece las siguientes actividades:

Requerimientos → Análisis → Diseño → Implementación → Pruebas.

El proceso solamente establece que esas actividades forman parte del desarrollo.

El ciclo de vida determina cómo se ejecutarán.

En cascada:

100 % requerimientos → 100 % análisis → 100 % diseño → 100 % implementación → 100 % pruebas.

En un ciclo iterativo:

Parte de los requerimientos → análisis → diseño → implementación → pruebas → versión funcional.

Luego se comienza una nueva iteración:

Nuevos requerimientos → análisis → diseño → implementación → pruebas → nueva versión.

Por esta razón, el ciclo de vida puede entenderse como aquello que indica cómo utilizar el proceso dentro de un proyecto determinado.

Relación con procesos definidos y empíricos

Los procesos definidos pueden utilizar ciclos de vida pertenecientes a cualquiera de los tres tipos generales:

- Secuencial.
- Iterativo/incremental.
- Recursivo.

En cambio, los procesos empíricos necesitan utilizar un ciclo de vida iterativo. Esto se debe a que el empirismo se basa en el aprendizaje obtenido a partir de la propia experiencia del equipo. Para generar esa experiencia rápidamente, se necesitan iteraciones pequeñas y frecuentes. El equipo desarrolla una parte del producto, obtiene software funcionando, recibe retroalimentación y utiliza esa información para ajustar el trabajo siguiente.

La lógica es:

Desarrollar → obtener resultado → recibir feedback → aprender → ajustar → volver a desarrollar.

Por este motivo, un proceso empírico no puede depender de un ciclo puramente secuencial. Si toda la retroalimentación se obtuviera únicamente al finalizar el proyecto, el equipo recibiría demasiado tarde la información necesaria para aprender y adaptarse. En consecuencia, la iteración es fundamental dentro de los procesos empíricos porque permite generar conocimiento rápidamente a partir de la experiencia real del equipo.

PROYECTO: es un esfuerzo realizado con el propósito de alcanzar un resultado específico y único dentro de un período de tiempo determinado. A diferencia de una actividad de producción continua, un proyecto posee un comienzo y un final definidos y persigue un objetivo concreto.

Para poder identificar correctamente si una actividad constituye un proyecto, deben analizarse principalmente sus características.

Resultado único: Esto significa que el resultado buscado no corresponde a una producción repetitiva o masiva. Si una actividad consiste simplemente en fabricar continuamente el mismo producto mediante una línea de producción, no se está frente a un proyecto.

Por ejemplo, el desarrollo de una aplicación específica para resolver una determinada necesidad constituye un proyecto porque se busca obtener un producto particular. En cambio, una línea industrial que fabrica continuamente el mismo producto corresponde a una actividad de producción.

Duración limitada: Esto significa que debe existir: Fecha de inicio → ejecución del proyecto → fecha de finalización.

La existencia de fechas planificadas no garantiza que estas se cumplan exactamente. Sin embargo, una de las responsabilidades de quien administra el proyecto consiste en intentar cumplir los plazos establecidos.

Si durante el desarrollo no resulta posible cumplirlos, deben identificarse y justificarse las causas y tomarse las acciones necesarias para controlar la situación.

Lo importante es que un proyecto necesariamente debe comenzar y, en algún momento, debe finalizar.

Esta característica permite diferenciarlo claramente de una línea de producción. Una línea productiva puede comenzar a funcionar y continuar durante un período indefinido, sin que desde el inicio exista necesariamente una fecha determinada para detenerla. Puede finalizar porque el producto deja de fabricarse, pero esa finalización no forma parte necesariamente de una duración previamente establecida como ocurre en un proyecto.

Objetivo definido: Todo proyecto debe poseer un objetivo definido. Este objetivo representa aquello que se pretende obtener como resultado de la realización del proyecto. Para que el objetivo sea adecuado debe cumplir dos características fundamentales:

- Ser claro.
- Ser alcanzable.

Objetivo claro: Un objetivo es claro cuando todas las personas involucradas comprenden de la misma manera qué resultado se pretende alcanzar. No debe existir ambigüedad respecto de aquello que debe obtenerse al finalizar el proyecto.

Por ejemplo, indicar únicamente que debe desarrollarse “una aplicación” no representa un objetivo suficientemente claro, ya que no especifica qué debe hacer la aplicación ni cuál es el resultado esperado.

En cambio, indicar que debe desarrollarse una billetera virtual que permita almacenar dinero, realizar pagos y determinadas operaciones establece con mayor precisión el resultado que se pretende alcanzar.

Objetivo alcanzable: Un objetivo es alcanzable cuando, considerando las condiciones existentes, el equipo posee una posibilidad real de lograrlo realizando el mejor trabajo posible. Para determinar si un objetivo es alcanzable deben considerarse factores como:

- tiempo disponible;
- cantidad de personas;
- experiencia del equipo;
- conocimientos técnicos;
- complejidad del producto;
- recursos disponibles.

Por lo tanto, un objetivo puede ser perfectamente claro y, al mismo tiempo, no ser alcanzable. Por ejemplo, puede establecerse claramente que un equipo debe desarrollar una billetera virtual completa, con pagos, validaciones e integración con diferentes servicios externos. Todos pueden comprender exactamente qué debe realizarse. Sin embargo, si el equipo dispone solamente de una semana y no posee suficientes recursos o experiencia, el objetivo puede resultar imposible de alcanzar dentro de las condiciones establecidas.

Diferencia entre objetivo claro y alcanzable: Los conceptos de claridad y posibilidad de cumplimiento no representan lo mismo.

- **Objetivo claro:** se comprende exactamente qué se quiere lograr.
- **Objetivo alcanzable:** existen condiciones razonables para poder lograrlo.

Por esta razón, pueden darse diferentes situaciones: Un objetivo puede ser claro pero no alcanzable cuando se comprende perfectamente qué debe hacerse, pero no existen suficientes recursos, tiempo o capacidad para conseguirlo.

También puede existir un objetivo potencialmente alcanzable pero poco claro cuando el equipo posee los conocimientos y recursos necesarios, pero no existe una definición precisa de aquello que debe entregar.

Por lo tanto, para que un proyecto pueda administrarse correctamente, su objetivo debe ser simultáneamente claro y alcanzable.

La “bala de plata” y la ilusión tecnológica

La idea de la **bala de plata** también se relaciona con la tendencia a pensar que una nueva herramienta puede resolver todos los problemas. Es similar a creer que, por tener un martillo, todo problema debe ser tratado como si fuera un clavo.

Cuando aparece una nueva tecnología, suele generarse la expectativa de que solucionará gran parte de las dificultades existentes. Esto ocurrió con la programación orientada a objetos y actualmente ocurre con la inteligencia artificial generativa.

Sin embargo, estas tecnologías actúan principalmente sobre la **complejidad accidental**, es decir, sobre la forma de implementar, representar o construir el software. No eliminan la **complejidad esencial**, que está vinculada con comprender el problema, definir correctamente los requerimientos, diseñar una solución y tomar decisiones.

Por eso, la inteligencia artificial puede ayudar a programar, generar código o automatizar tareas, pero no reemplaza completamente el análisis y el razonamiento necesarios para construir correctamente un sistema.

El software como actividad humana

Frederick Brooks también plantea que el desarrollo de software es una actividad fuertemente dependiente de las personas. Gran parte del costo de un proyecto está relacionada con el esfuerzo humano necesario para analizar, diseñar, implementar y probar el sistema.

Esto explica por qué el trabajo en software no puede tratarse como una producción industrial común. Las personas no son recursos completamente intercambiables ni su trabajo puede dividirse indefinidamente.

Agregar personas no siempre acelera un proyecto

Una idea importante de Brooks es que **agregar personas a un proyecto atrasado puede atrasarlo todavía más**.

Una persona nueva necesita tiempo para:

- comprender el proyecto;
- aprender su funcionamiento;
- conocer las decisiones ya tomadas;
- comunicarse con los demás integrantes;
- coordinar su trabajo.

Además, los miembros que ya estaban trabajando deben dedicar tiempo a incorporar y capacitar a las nuevas personas.

Por eso, aumentar la cantidad de integrantes también aumenta la complejidad de comunicación y coordinación.

Un ejemplo clásico es que un embarazo requiere aproximadamente nueve meses: agregar más personas no permite obtener el resultado en menos tiempo. Existen tareas que poseen restricciones secuenciales y no pueden acelerarse simplemente agregando recursos.

Horas extra y agotamiento

Otra solución frecuente frente a un proyecto atrasado consiste en aumentar las horas de trabajo.

Esto puede funcionar durante períodos muy breves, pero no es sostenible en trabajos intelectuales. Cuando una persona trabaja demasiado, duerme poco o se encuentra agotada, aumenta la probabilidad de cometer errores.

Como consecuencia, parte del tiempo que aparentemente se gana debe utilizarse posteriormente para detectar y corregir esos errores. Por lo tanto, mantener durante mucho tiempo jornadas excesivas puede terminar empeorando el problema.

Integridad conceptual

Otro principio importante es la **integridad conceptual**.

Aunque muchas personas participen en el desarrollo de un sistema, debe existir una visión coherente sobre cómo debe funcionar y qué características debe tener.

Las decisiones principales no deberían convertirse en una suma desordenada de opiniones independientes. Puede existir participación de muchas personas, pero debe haber coordinación y una dirección clara que mantenga la coherencia general del producto.

El ejemplo puede compararse con una intervención quirúrgica: muchas personas participan y colaboran, pero debe existir una conducción clara para evitar decisiones contradictorias.

Evitar complejidad innecesaria

También existe una tendencia a agregar funcionalidades simplemente porque técnicamente pueden incorporarse.

Esto puede expresarse como:

“Ya que estamos, agreguemos también esto”.

El problema es que muchas de esas funcionalidades pueden no haber sido solicitadas por el usuario o incluso nunca llegar a utilizarse.

Por eso aparece el principio **YAGNI — You Aren’t Gonna Need It**, que plantea que no debería desarrollarse una funcionalidad solamente porque podría ser útil en el futuro.

La idea es construir aquello que realmente se necesita y evitar agregar complejidad innecesaria.

Calidad del producto, compromisos y documentación

La **calidad del software no debería ser negociable**. Los conocimientos relacionados con diseño, cohesión, acoplamiento, arquitectura y buenas prácticas apuntan a construir un producto de calidad. Por otro lado, la gestión del proyecto se ocupa de aspectos como tiempos, estimaciones, recursos y compromisos. Ambas dimensiones son diferentes, pero están directamente relacionadas, porque las decisiones tomadas durante la gestión terminan afectando al producto final.

Por ejemplo, si un equipo se compromete a entregar un producto en la mitad del tiempo realmente necesario, probablemente alguna característica del proyecto termine viéndose afectada. Sin embargo, la calidad no debería ser aquello que se sacrifique para cumplir una fecha.

Por esta razón, lo que debe negociarse son los **compromisos**. Si una estimación indica que determinado trabajo requiere cierto tiempo, no debería modificarse artificialmente para adaptarse a una fecha deseada. En cambio, pueden negociarse aspectos como el alcance, las funcionalidades incluidas, las prioridades o la fecha de entrega.

La documentación también forma parte de la calidad

En todo proyecto debe existir algún registro de las decisiones importantes tomadas respecto del producto. No es fundamental el formato utilizado: puede tratarse de documentos, diagramas, pizarras, repositorios u otros medios.

Lo importante es registrar, al menos:

- qué debe hacer el producto, es decir, sus requerimientos;
- cómo se decidió resolver esos requerimientos;
- las principales decisiones de diseño y arquitectura;
- los cambios relevantes realizados durante el desarrollo.

La documentación es importante porque **escribir obliga a expresar las ideas con mayor precisión**. Durante una conversación puede parecer que un concepto está claro, pero al intentar escribirlo pueden aparecer dudas, contradicciones o aspectos que todavía no fueron definidos.

Por lo tanto, documentar no consiste solamente en dejar información para el futuro. También funciona como una herramienta para detectar aquello que el equipo todavía no comprende completamente.

Relación entre gestión y producto

La gestión del proyecto y la calidad técnica del producto no pueden analizarse de manera completamente independiente.

Una mala decisión de gestión puede terminar generando consecuencias técnicas. Por ejemplo:

plazo irreal → presión sobre el equipo → reducción de pruebas o diseño → más errores → menor calidad

Por eso, administrar correctamente un proyecto no significa solamente intentar cumplir una fecha, sino establecer compromisos realistas sin deteriorar la calidad del producto.

¿Por qué Scrum no se plantea para solo dos personas?

Scrum requiere un **Scrum Team** con las responsabilidades necesarias para que exista autogestión, desarrollo del producto, coordinación y control del trabajo. El equipo contempla tres responsabilidades fundamentales:

- **Product Owner:** define y prioriza qué aporta valor al producto.
- **Scrum Master:** ayuda a que Scrum se aplique correctamente y facilita el funcionamiento del equipo.
- **Developers:** realizan el trabajo necesario para construir el incremento del producto.

La idea no es simplemente “tener cierta cantidad de personas”, sino contar con **suficientes integrantes y capacidades para que el equipo pueda ser multifuncional y autogestionado**.

Con únicamente dos personas resulta difícil mantener correctamente estas responsabilidades, distribuir el trabajo y disponer de las distintas habilidades necesarias para producir un incremento funcional en cada Sprint.

Además, Scrum busca equipos **pequeños pero suficientemente completos**: agregar demasiadas personas aumenta los problemas de comunicación, pero tener demasiado pocas puede impedir que el equipo posea todas las capacidades necesarias.

Idea para recordar:

Scrum busca un equipo lo suficientemente pequeño para comunicarse fácilmente, pero lo suficientemente grande y completo para poder construir el producto y cumplir todas las responsabilidades del framework.

Importante: la **Scrum Guide actual no establece literalmente un mínimo de personas**, aunque sí define las tres responsabilidades anteriores. Si en la materia les indican un número mínimo concreto, conviene usar **el número establecido por la bibliografía o versión de Scrum que utiliza la cátedra**, porque versiones anteriores del framework sí establecían tamaños más específicos.

Administración de proyectos y equilibrio entre alcance, tiempo y costo

Para que un proyecto de software sea considerado exitoso no alcanza con terminar el producto. También es necesario que el software tenga calidad, que satisfaga las necesidades y expectativas del cliente y que, al mismo tiempo, el proyecto sea viable desde el punto de vista económico. Esto implica que debe existir una relación adecuada entre lo que se entrega, el tiempo disponible y los costos involucrados.

Administración de proyectos

La administración de proyectos se compone principalmente de dos grandes actividades: la planificación y el seguimiento o supervisión. Primero se debe establecer un plan y luego controlar cómo avanza realmente el proyecto respecto de ese plan.

Es posible planificar y después no realizar seguimiento, aunque en ese caso el plan pierde rápidamente su utilidad. En cambio, no es posible realizar un seguimiento correcto si previamente no se planificó, porque para controlar hace falta comparar la situación real con una referencia. Esa referencia es justamente el plan.

Las tres dimensiones del proyecto

Todo proyecto se mueve principalmente sobre tres dimensiones: alcance, tiempo y costo. El alcance representa qué se debe construir y está directamente relacionado con los requerimientos y funcionalidades del producto. El tiempo representa cuánto se dispone para realizar el proyecto y cuándo debe entregarse. El costo surge de los recursos necesarios y del tiempo durante el cual esos recursos son utilizados.

Cuando se habla de recursos, especialmente en proyectos de software, es más correcto pensar en el trabajo realizado por las personas y no simplemente considerar a las personas como recursos intercambiables.

Equilibrio entre alcance, tiempo y costo

La gestión de proyectos busca encontrar el mejor equilibrio posible entre alcance, tiempo y costo sin afectar la calidad. Estas tres dimensiones están relacionadas entre sí, por lo que modificar una suele producir consecuencias sobre las demás.

Por ejemplo, si aumenta el alcance pero se mantiene el mismo tiempo, probablemente sea necesario aumentar los recursos y, por lo tanto, el costo. Si se reduce el tiempo disponible manteniendo el mismo alcance, también puede ser necesario aumentar los recursos. El problema aparece cuando se intenta mantener un alcance muy grande, un tiempo reducido y un costo bajo al mismo tiempo, porque alguna variable termina afectada y muchas veces la primera perjudicada es la calidad.

La calidad no debería negociarse

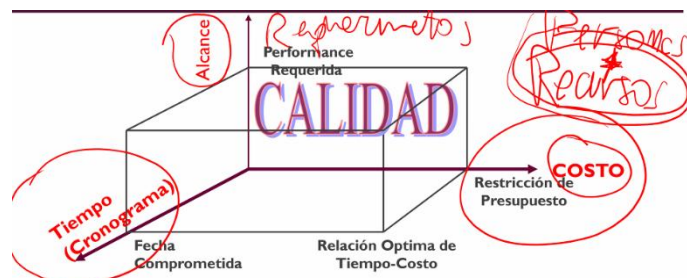
La calidad del producto de software está relacionada con su capacidad para satisfacer las necesidades y expectativas del cliente. Por eso, cuando aparecen restricciones de tiempo, costo o alcance, la calidad no debería utilizarse como variable de ajuste.

Si un proyecto no puede realizarse en las condiciones inicialmente planteadas, deberían revisarse y negociarse los compromisos. Puede modificarse el alcance, el tiempo disponible o los recursos, pero no debería aceptarse deliberadamente una reducción de la calidad solo para cumplir una fecha.

El papel del cliente

La principal variable que debería definir el cliente es el alcance, porque es quien conoce qué necesita obtener del producto. El cliente debe expresar qué problema quiere resolver, qué necesita y qué espera del sistema.

Sin embargo, lo que el cliente dice que quiere no siempre coincide exactamente con lo que realmente necesita. Por eso, el equipo debe analizar esos pedidos y trabajar junto con el cliente hasta llegar a una definición adecuada de los requerimientos.



A partir de ese alcance, el equipo técnico y quienes administran el proyecto deben determinar cuánto tiempo y qué recursos son necesarios para realizarlo.

Restricciones del proyecto

En algunos proyectos existen restricciones que no pueden modificarse. Por ejemplo, si un software debe utilizarse en una elección, la fecha de entrega está condicionada por el día de la elección. Entregarlo después ya no tendría sentido.

Cuando una dimensión queda fija, las demás deben poder ajustarse. Si el tiempo no puede modificarse, probablemente haya que negociar el alcance o los recursos necesarios.

Para que una persona pueda realmente administrar un proyecto, debe tener cierto margen para tomar decisiones. Si el cliente determina completamente qué se debe hacer, cuánto se va a pagar y cuándo debe entregarse, prácticamente no queda espacio para gestionar el proyecto.

El líder de proyecto y el equipo en la gestión tradicional

En la gestión tradicional existe un **líder de proyecto**, responsable de estimar, elaborar el plan y tomar las principales decisiones relacionadas con el proyecto. Estas decisiones se vinculan especialmente con las tres dimensiones

fundamentales: alcance, tiempo y costo. Además, el líder coordina al conjunto de personas que forman el **equipo de proyecto**.

El equipo tradicional suele organizarse de manera funcional y especializada. Puede estar formado por programadores, analistas funcionales, arquitectos o líderes técnicos, testers, responsables de calidad, responsables de configuración y especialistas en experiencia de usuario, entre otros. Cada integrante suele desempeñar una función específica según su especialidad.

El líder también suele actuar como vínculo entre el equipo y los demás **stakeholders**, es decir, todas las personas interesadas o afectadas por el proyecto. Por ejemplo, mantiene la comunicación con clientes, gerentes, responsables funcionales y otros niveles de la organización. De esta manera, funciona como una interfaz entre el equipo de trabajo y el resto de los involucrados.

Diferencia con los equipos ágiles

La organización de un equipo ágil es diferente porque se basa principalmente en la **autogestión**. En la gestión tradicional existe un líder que distribuye tareas, realiza estimaciones y controla el avance. En cambio, en un enfoque ágil son las propias personas que realizan el trabajo quienes participan de las estimaciones, asumen compromisos y organizan cómo llevarlos adelante.

Por esta razón, la autogestión es uno de los aspectos fundamentales del agilismo. No existe un jefe que indique permanentemente qué debe hacer cada integrante o cuándo debe terminarlo. Cada miembro asume un compromiso y también la responsabilidad de cumplirlo.

Equipos especializados y equipos multidisciplinarios

En la gestión tradicional los equipos suelen organizarse de manera **funcional**, es decir, cada integrante se especializa en una determinada actividad. Una persona puede dedicarse exclusivamente al análisis funcional, otra al desarrollo, otra al testing y otra al diseño.

Los enfoques ágiles, en cambio, buscan formar **equipos multidisciplinarios**. Esto significa que el equipo debe contar con todas las capacidades necesarias para llevar una funcionalidad desde su inicio hasta un estado completamente terminado.

La idea no se limita a que una persona pueda programar tanto frontend como backend. Desde la perspectiva ágil, también debería poder participar en la comprensión de requerimientos, realizar las tareas de diseño necesarias, desarrollar, probar, documentar y dejar una funcionalidad preparada para ser evaluada por el cliente.

En términos teóricos, un integrante debería poder comprometerse con una historia de usuario, comenzarla y llevarla hasta su finalización antes de asumir otra. Sin embargo, llevar este concepto completamente a la práctica puede resultar difícil en organizaciones que históricamente contrataron personas con funciones muy especializadas.

Enfoques híbridos

No todo elemento de la gestión tradicional y del agilismo puede mezclarse libremente. Algunas **prácticas** pueden combinarse siempre que sean compatibles con la filosofía del enfoque utilizado.

Por ejemplo, un proceso empírico necesita un ciclo de vida iterativo porque requiere obtener experiencia y retroalimentación frecuente. Por lo tanto, combinar un proceso empírico con un ciclo de vida completamente secuencial generaría una contradicción conceptual.

La elección de prácticas debe realizarse analizando si estas son compatibles con los principios del enfoque utilizado. Cuando existe compatibilidad, pueden construirse modelos híbridos.

El plan de proyecto

Dentro de la gestión tradicional, el **plan de proyecto** funciona como una hoja de ruta que establece cómo se pretende alcanzar el objetivo.

Un plan debe responder cuestiones relacionadas con el objetivo, el alcance del proyecto, el proceso que se utilizará, el ciclo de vida elegido, las estimaciones, los riesgos, la asignación de recursos y el cronograma.

El cronograma, por ejemplo un diagrama de Gantt, constituye solamente una parte de la planificación. No puede considerarse por sí solo un plan de proyecto. Generalmente se construye después de haber determinado qué debe hacerse, cuáles son los riesgos, qué recursos existen y cuánto esfuerzo requerirá el proyecto.

Alcance del proyecto y alcance del producto

Es fundamental diferenciar el **alcance del producto** del **alcance del proyecto**.

El alcance del producto define **qué debe ser o hacer el producto**. En software está representado principalmente por sus requerimientos funcionales y no funcionales y se documenta en una especificación o documento de requerimientos.

El alcance del proyecto, en cambio, representa **todo el trabajo necesario para alcanzar el objetivo del proyecto**. Se especifica en el plan del proyecto.

Por ejemplo, si se desea desarrollar una aplicación para gestionar una agenda personal, el alcance del producto puede incluir registrar compromisos, organizar actividades y generar recordatorios. El alcance del proyecto incluiría las actividades necesarias para conseguir ese producto: analizar requerimientos, elegir tecnologías, diseñar la arquitectura, desarrollar, probar y realizar las demás tareas necesarias.

Un mismo producto puede dar lugar a proyectos diferentes

El alcance del producto y el del proyecto están relacionados, pero no son iguales.

Por ejemplo, puede mantenerse exactamente la misma idea de una aplicación de agenda, pero plantearse un proyecto cuyo objetivo sea únicamente producir un **documento completo de requerimientos** para determinar posteriormente si conviene desarrollar el software.

En ese caso, el alcance del producto continúa siendo el mismo, pero cambia el alcance del proyecto. Ya no sería necesario desarrollar, implementar y probar el sistema, sino solamente realizar el trabajo necesario para obtener la especificación de requerimientos.

Esto demuestra que una modificación del alcance del proyecto no necesariamente implica una modificación del alcance del producto.

Elección del proceso y del ciclo de vida

Una vez establecidos el objetivo y el alcance, el plan debe determinar qué **proceso** y qué **ciclo de vida** se utilizarán.

El proceso funciona como referencia para saber qué actividades pueden ser necesarias para desarrollar software. Luego debe adaptarse al contexto específico del proyecto, determinando qué actividades se realizarán, qué roles existirán y quién asumirá cada uno.

El ciclo de vida determina cómo se distribuirán esas actividades a lo largo del proyecto. Para proyectos de software suele resultar conveniente utilizar ciclos iterativos debido a la aparición de **requerimientos emergentes**, es decir, requerimientos que solamente se descubren cuando el cliente observa versiones concretas del producto.

Estimaciones en la gestión tradicional

En la gestión tradicional las estimaciones suelen buscar valores absolutos y un nivel importante de precisión. Se intenta determinar cuánto trabajo existe, cuánto esfuerzo será necesario, cuándo podrá completarse y cuánto costará.

El orden de estimación es importante:

Tamaño → Esfuerzo → Calendario → Costo

Cada estimación depende de la anterior y responde una pregunta diferente.

Estimación del tamaño

Primero se estima el **tamaño del producto**, es decir, qué tan grande es aquello que debe construirse.

Esta estimación debe ser independiente de quién realizará el trabajo, de cuánto dinero tenga disponible el cliente o de cuánto tiempo quiera dedicar al proyecto. Busca exclusivamente determinar la magnitud del producto.

En enfoques tradicionales pueden utilizarse elementos como requerimientos o casos de uso clasificados según su complejidad. Las líneas de código no resultan adecuadas para estimaciones tempranas porque al comenzar el proyecto todavía no se sabe cuánto código será necesario escribir.

Estimación del esfuerzo

A partir del tamaño se estima el **esfuerzo necesario** para desarrollar el producto.

El esfuerzo se expresa normalmente en **horas-persona**. Representa la cantidad total de trabajo necesaria suponiendo conceptualmente que una persona realiza una tarea a la vez.

Por ejemplo, una estimación podría determinar que un proyecto requiere 2.000 horas-persona. Esto todavía no indica cuánto tiempo calendario tardará el proyecto, sino únicamente la cantidad total de trabajo necesaria.

Estimación del calendario

Una vez conocido el esfuerzo, puede determinarse el **calendario**. En esta instancia se transforma la cantidad de horas de trabajo en una estimación de cuándo podría finalizar el proyecto.

Para ello deben considerarse la cantidad de personas disponibles, qué actividades pueden realizarse en paralelo, cuántas horas trabaja cada persona por día, cuántos días por semana trabaja y qué porcentaje de su tiempo puede dedicar al proyecto.

Por lo tanto:

Esfuerzo = cuánto trabajo existe.

Calendario = cuánto tiempo real llevará realizar ese trabajo.

Estimación del costo

Finalmente se determina el **costo del proyecto**, porque depende de las estimaciones anteriores.

Para calcularlo debe conocerse cuánto cuesta cada hora de trabajo y cuántas horas dedicará cada persona. A eso se agregan otros costos relacionados con infraestructura, conectividad, licencias, herramientas, alquileres y demás elementos utilizados durante el proyecto.

Por esta razón, el costo se calcula al final de la cadena de estimaciones. Si cambia la cantidad de personas, las horas trabajadas o el calendario, también puede modificarse el costo.

Secuencia general de la planificación tradicional

La lógica puede resumirse de la siguiente manera:

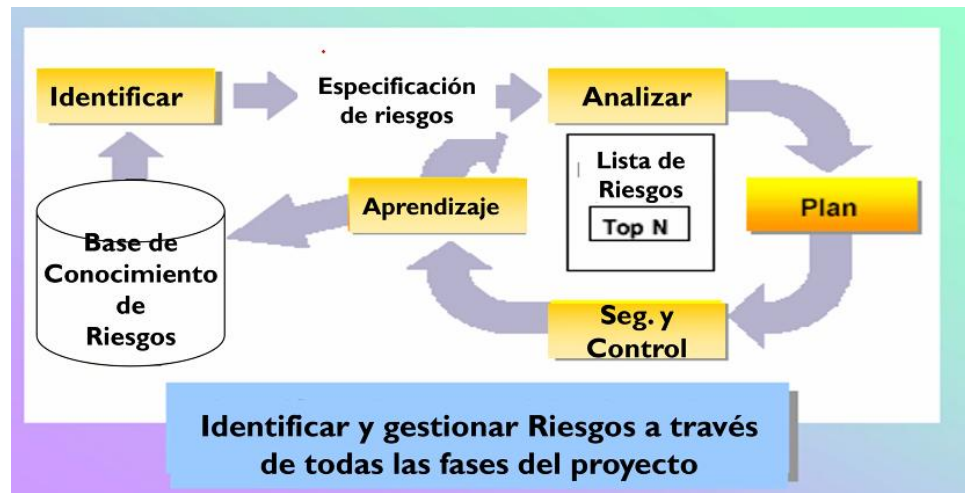
Objetivo y alcance → Proceso y ciclo de vida → Estimación del tamaño → Estimación del esfuerzo → Calendario → Costo → Cronograma

El cronograma aparece hacia el final porque solamente puede organizarse correctamente el trabajo cuando ya se conoce **qué debe hacerse, cuánto trabajo representa, con qué recursos se cuenta y cuánto tiempo puede requerir.**

GESTIÓN DE RIESGOS

Después de realizar las estimaciones del proyecto, uno de los aspectos que debe analizarse es la **gestión de riesgos**. Un riesgo es una situación incierta que puede ocurrir o no y que, en caso de producirse, puede generar un impacto negativo sobre el proyecto.

La característica fundamental de un riesgo es que posee una **probabilidad de ocurrencia**. Por esta razón, no debe confundirse con un problema. Un problema es una situación que ya existe o de la cual se tiene certeza, mientras que un riesgo todavía puede ocurrir o no.



Riesgo y problema

La diferencia principal entre ambos conceptos es la incertidumbre. Si existe alguna posibilidad de que una situación ocurra, se trata de un riesgo. Si se sabe con certeza que ocurrirá o ya está ocurriendo, entonces se trata de un problema.

Por ejemplo, si existe la posibilidad de que el proyecto no llegue a terminarse dentro del plazo previsto, puede considerarse un riesgo. Sin embargo, si ya se sabe que el trabajo necesita seis meses y se asumió un compromiso de tres meses, entonces la falta de tiempo ya no debería considerarse un riesgo, sino un problema que debe ser solucionado.

Por eso, puede decirse que un riesgo es, en cierta forma, **un problema que podría llegar a ocurrir**.

Exposición al riesgo

Para evaluar la importancia de un riesgo se utiliza el concepto de **exposición al riesgo**. Esta se obtiene considerando dos dimensiones: la probabilidad de ocurrencia y el impacto.

Exposición = Probabilidad × Impacto

La **probabilidad** representa qué tan posible es que el riesgo ocurra. El **impacto** representa cuánto daño produciría sobre el proyecto si efectivamente ocurriera.

Este cálculo permite transformar una percepción subjetiva en un valor que facilite la comparación y priorización de diferentes riesgos.

Por ejemplo, un riesgo puede tener una probabilidad muy baja pero un impacto extremadamente alto. Otro puede tener una probabilidad elevada pero consecuencias menores. Mediante la exposición se puede determinar cuál de ellos requiere mayor atención.

Gestión proactiva de riesgos

Una gestión adecuada debe ser **proactiva**, lo que significa identificar y analizar los riesgos antes de que se conviertan en problemas.

Dentro del plan del proyecto deberían identificarse los riesgos, estimarse sus probabilidades e impactos y calcularse su exposición. A partir de esto deben decidirse las acciones que se tomarán frente a cada uno.

La gestión proactiva permite anticiparse a posibles dificultades en lugar de actuar únicamente cuando el problema ya ocurrió. Además, genera experiencia y conocimiento que pueden ser aprovechados tanto en el mismo proyecto como en proyectos futuros.

Mitigación y contingencia

Frente a los riesgos pueden definirse principalmente acciones de **mitigación y contingencia**.

La mitigación consiste en realizar acciones previamente para **reducir la probabilidad de que el riesgo ocurra o disminuir sus consecuencias**. En cambio, la contingencia establece qué se hará **si finalmente el riesgo se materializa**.

Por ejemplo, si existe el riesgo de que una persona clave abandone el proyecto, una acción de mitigación podría ser compartir el conocimiento y mantener documentación actualizada. El plan de contingencia podría establecer cómo se redistribuirá su trabajo o quién asumirá sus responsabilidades si efectivamente abandona el equipo.

Riesgos frecuentes en proyectos de software

Uno de los principales riesgos está relacionado con los **requerimientos**. Comprender incorrectamente qué necesita el cliente, que aparezcan cambios importantes o que los requerimientos estén incompletos puede afectar significativamente el proyecto. Esto se relaciona con una de las dificultades esenciales del desarrollo de software: determinar correctamente qué software debe construirse.

Otro riesgo frecuente está relacionado con las **personas**. Puede ocurrir que no se disponga de las capacidades necesarias o que personas importantes abandonen el proyecto antes de su finalización. La pérdida de integrantes que poseen conocimientos críticos puede afectar tiempos, costos y calidad.

También existe el **riesgo tecnológico**. Los proyectos de software trabajan permanentemente con tecnología y, cuanto más nueva o menos probada sea una herramienta, mayor puede ser la incertidumbre respecto de su funcionamiento. Una tecnología puede no ofrecer las capacidades esperadas, presentar incompatibilidades o generar dificultades no previstas.

Finalmente, también son habituales los riesgos relacionados con **estimaciones incorrectas**, ya sea porque se subestima el tamaño, el esfuerzo, el tiempo o los recursos necesarios.

Idea principal

La gestión de riesgos consiste en **identificar anticipadamente situaciones inciertas que podrían perjudicar al proyecto, medir su importancia y preparar acciones para enfrentarlas**.

La relación fundamental es:

Riesgo = existe incertidumbre.

Problema = existe certeza o ya ocurrió.

Exposición = Probabilidad × Impacto.

El objetivo no es eliminar completamente la incertidumbre, sino conocerla y prepararse para responder de la mejor manera posible.